

**"Evaluación integradora - Programación
para Inteligencia Artificial - Primera
parte".**

Gentile, Agustín Raúl (Dni: 32709735) (Legajo: 199433)

Estudiante: Gentile, Agustín Raúl

DNI: 32,7097,735

Fecha entrega: 4/09/2026

Legajo: 199433

Parte A: Teorico-Práctico

1. Estructuras de control

a) Explique cuál es la diferencia entre una estructura condicional (if/elif/else) y una estructura repetitiva (while/for). De un ejemplo concreto de cuándo usaría cada una.

La principal diferencia entre estas dos estructuras es que la “estructura condicional” en caso de que se cumpla la condición establecida se repite una sola vez el bloque debajo de ella, la estructura repetitiva en la mayoría de los casos ejecuta 1 o n veces.

Las estructuras condicionales se usan cuando quieres tomar un camino u otro dependiendo de la evaluación de una condición (de ahí el nombre de “condicional”); es como decir si pasa esto ejecuta esto.

Las estructuras repetitivas son usadas cuando se quiere ejecutar funcionalidad repetidamente.

Ejemplo: Dada una lista de personas con datos de apellido, nombre, edad. Mostrar los datos de las personas mayores a 18 años.

En este ejemplo para recorrer a todas las personas se usan las estructuras repetitivas; la condicional nos daría la certeza de solo mostrar los datos de aquellas personas que cumplen la condición de ser mayores a 18.

b) ¿Qué diferencia hay entre un bucle while y un bucle for? ¿En qué situaciones es más conveniente usar cada uno?

La diferencia principal radica que en la estructura while repite la secuencia mientras una condición se cumpla, en cambio el bucle for repite la secuencia una determinada cantidad de veces.

El bucle for se usa cuando sabes cuántas veces repetir, conoces de antemano la cantidad de repeticiones, recorrer colecciones, lista, array. ; en cambio en la estructura while no sabes cuántas veces vas a repetir y el ciclo se repite hasta que se cumpla una condición, para o caer en un bucle infinito esta condición debe cumplirse.

2. Funciones en python

a). Explique qué son los parámetros de una función y qué es el valor de retorno. ¿Por qué es importante organizar el código en funciones?

Los parámetros son los valores que recibe una función desde el lugar en que es invocada; es la forma en que podemos ingresar datos a la función desde afuera. El retorno son

datos que la función devuelve para retornarlos al exterior (es decir al lugar donde fue invocada).

Las funciones son importantes por que permiten la segmentación de los programas, modularizar un sistema, es decir dividirlo en partes más pequeñas, cada una encargada de resolver un problema en particular; así se puede tener un programa más limpio, ordenado, fácil de mantener y testeable. Cada función es responsable de una única responsabilidad. Lleva a cabo una única tarea.

3. Estructuras de datos en *python*

a) . Python ofrece diferentes estructuras de datos: listas, tuplas, diccionarios y conjuntos. Para cada una de las siguientes situaciones, indique qué estructura usaría y justifique brevemente:

1. Almacenar información de un estudiante con nombre, legajo y promedio.

2. Guardar una colección de IDs únicos sin duplicados ni orden específico.

3. Mantener una secuencia ordenada de calificaciones que se pueda modificar.

1. Almacenaría la información del estudiante en un diccionario, con claves pares => valores. Creo que es la estructura especial en estos casos; ya que la información queda más clara al tener una estructura hashable; sabes con las llaves que dato se almacena

2. Para almacenar la colección de ID, al ser únicos, no importa el orden de los mismos y evitar duplicados, la mejor estructura para esto son los conjuntos = SET , que es la estructura de datos que python ofrece para manejar colección de datos no ordenados y sin repetir.

3. Para mantener una colección de datos ordenada (secuencia), la cual se puede modificar de usar las lista LIST. , ya que nos permite almacenar una colección de datos ordenada, modificable y se pueden operar sobre los mismos.

4. Pilas y Colas

1. Explique qué significa LIFO y qué significa FIFO. De un ejemplo de cada uno.

LIFO: Es un concepto que determina que el elemento primero en entrar es el último en salir, o mejor dicho los más viejos son los últimos que se obtienen. Después la estructura de datos que lo implemente debe respetar este concepto. Primero en entrar último en salir. Un ejemplo sistemas de navegación, sistemas de visitas, etc.

FIFO: Es un concepto que determina que el elemento primero en entrar es el primero en salir, o mejor dicho los más viejos son los primeros que se obtienen. Cola de embalaje donde se debe respetar la cadena sin saltarse los procesos, sistema de atención de llamadas (si no tienen peso las mismas y el orden es primero en ser recibida, primero en ser atendida)

2. Si implementamos una cola en Python usando una lista, ¿cómo agregaríamos elementos y cómo los sacaríamos para respetar el principio FIFO?

Gentile, Agustín Raúl (Dni: 32709735) (Legajo: 199433)

Para representar un cola usando listas, y poder agregar o sacar informacion, utilizo las funciones que nos provee las listas que son append la cual agrega un elemento al final, y la funcion pop(0) para obtener y sacar el primer elemento de la secuencia.

Ejemplo: cola = []

cola.append(10) => ahora cola es cola = [10]

cola.append(33) => ahora cola es cola = [10, 33]

cola.append(22) => ahora cola es cola = [10, 33, 22]

nro = cola.pop(0) => ahora cola es cola = [33, 22]

Parte B: Sistema Gestión Estudiantil

El sistema fue desarrollado siguiendo los principios de la modularización, dividiendo al sistema en funciones independientes, encargada cada una en cumplir un único objetivo. Los alumnos son representados con un diccionario que mantiene llaves => valores, es una estructura clara la cual permite acceder a la información mediante clave. El conjunto de alumnos se almacena en una lista, la cual considero que es la estructura ideal para colecciones de datos. Usamos una cola, para la atención de alumnos; ya que se sigue los principios lógicos de FIFO, los alumnos son atendidos en orden de llegada.

Se implementan las funcionalidades para la inserción de alumnos, existencia de ellos mismos por número de legajo, y búsqueda con retorno de posición, la cual ayuda a recuperar la información del alumno y a acceder a su contenido.

Gentile, Agustín Raúl (Dni: 32709735) (Legajo: 199433)

```
""" INCIO DE FUNCIONALIDAD PARA COLAS """
def crearCola():
    return []

def esta_vacia(cola):
    return len(cola) == 0

def enqueue(cola, elemento):
    cola.append(elemento)

def dequeue(cola):
    if not esta_vacia(cola):
        return cola.pop(0)
    return None

def ver_frente(cola):
    if not esta_vacia(cola):
        return cola[0]
    return None

def tamaño(cola):
    return len(cola)

""" INCIO DE FUNCIONALIDAD PARA COLAS """

""" INCIO DE FUNCIONALIDAD estudiantes """

# Comprueba si el nro de legajo existe en la lista de estudiante
def existe_legajo(lista_estudiantes, legajo):
    for estudiante in lista_estudiantes:
        if estudiante["legajo"] == legajo:
            return True
    return False

# Busca si existe un estudiante (pasando su nro de legajo) en la lista.
# Si existe devuelve su posición en la lista caso contrario None
def buscar_x_legajo(lista_estudiantes, legajo):
    for indice, estudiante in enumerate(lista_estudiantes):
        if estudiante["legajo"] == legajo:
            return indice
    return None

# verifica que una nota este en el rango entre 0 y 10, devuelve valor logico de
# acuerdo a si cumple la condicion.
def nota_valida(nota):
    return bool(nota > 0 and nota <= 10)

# crea un estudiante, con nombre, legajo, calificaciones y promedio
def crear_estudiante(nombre, legajo):
    return {
        "nombre": nombre,
        "legajo": legajo,
        "calificaciones": [],
        "promedio": 0.0 #por defecto se coloca en 0
    }

def ordenar_estudiantes_por_promedio(lista_estudiantes):
    n = len(lista_estudiantes)
    for i in range(n - 1):
```

Gentile, Agustín Raúl (Dni: 32709735) (Legajo: 199433)

```
#agarro el primero y lo compara que el resto, despues el segundo y asi
sucesivamente, se van comparando pares de hermano
for j in range(n - 1 - i):
    # Compra hermanos y los rotas trasladando el menor hacia el fondo,
    if lista_estudiantes[j]["promedio"] > lista_estudiantes[j + 1]
["promedio"]]:
    temporal = lista_estudiantes[j]
    lista_estudiantes[j] = lista_estudiantes[j + 1]
    lista_estudiantes[j + 1] = temporal

""" muestra la informacion de todos los estudiante de la lista"""
def mostrar_estudiante(lista_estudiantes):
    ordenar_estudiantes_por_promedio(lista_estudiantes)
    for estudiante in lista_estudiantes:
        print(f"Legajo: {estudiante['legajo']}, Nombre: {estudiante['nombre']},
Promedio: {estudiante['promedio']}, cantidad de calificaciones
{len(estudiante['calificaciones'])}")

""" Pide indormacion al usuario para crear un estudiante y lo agrega a la lista
de estudiantes """
def ingresar_estudiante(lista_estudiantes):
    nombre = input("Ingrese el nombre del estudiante: ")
    while True: # pedimos legajo hasta que ingrese uno que o exista
        legajo = input("Ingrese el legajo del estudiante: ")
        if existe_legajo(lista_estudiantes, legajo):
            print(f"El legajo {legajo} ya está asignado a otro estudiante.
Ingrese un legajo diferente.")
        else:
            break
    lista_estudiantes.append(crear_estudiante(nombre, legajo))

""" Calcula el promedio de una lista de calificaciones. Recibe como entrada un
list cuyas enttrdas son float , pertenecinetes a las notas"""
def calcular_promedio(estudiante):
    if len(estudiante["calificaciones"]) == 0:
        return 0.0
    return sum(estudiante["calificaciones"]) / len(estudiante["calificaciones"])

""" permire registrar un nota a un determinado estudiante, para esto pide datos
al usuario"""
def registrar_nota(lista_estudiantes):
    print("Lista de estudiantes:")
    mostrar_estudiante(lista_estudiantes)

    while True:
        nro_legajo = input("Ingrese el nro de legajo a ingresar nota:")
        pos_estudiante = buscar_x_legajo(lista_estudiantes, nro_legajo)
        if pos_estudiante is not None:
            break
        else:
            print(f"No se encontró un estudiante con el legajo {nro_legajo}.
Intente nuevamente.")

    while True:
        nueva_nota = input("Ingrese la nota obtenida, debe estar entre 0 y 10 :
")
```

```
        if(nota_valida(float(nueva_nota))): # asumimos que coloca un nro; sino
hay que gestionar lso errores capturando bloque try ctach
            break

    lista_estudiantes[pos_estudiante]
["calificaciones"].append(float(nueva_nota))
    lista_estudiantes[pos_estudiante]["promedio"] =
calcular_promedio(lista_estudiantes[pos_estudiante])

def crear_lista_estudiante():
    return []

""" agrega un determinado estudiante a la cola de atencio, la cola mantiene solo
el legajo,
acá se puedo mejorar marcando a un estudiante si ya esta en la cola para
prevebir duplicados,
ya que supuestamente solo puede estar una vez en la cola"""
def agregar_estudiante_cola(cola_espera, lista_estudiantes):
    print("Lista de estudiantes:")
    mostrar_estudiante(lista_estudiantes)

    while True:
        nro_legajo = input("Ingrese el nro de legajo a agregar a la cola de
consultas:")
        pos_estudiante = buscar_x_legajo(lista_estudiantes, nro_legajo)
        if pos_estudiante is not None:
            break
        else:
            print(f"No se encontró un estudiante con el legajo {nro_legajo}.
Intente nuevamente.")

    estudiante = lista_estudiantes[pos_estudiante]
    enqueue(cola_espera, estudiante["legajo"])
    print(f"Estudiante {estudiante['nombre']} agregado a la cola de consultas.")

def atender_consulta(cola_espera, lista_estudiantes):
    if esta_vacia(cola_espera):
        print("No hay estudiantes en la cola de consultas.")
        return

    legajo_atender = dequeue(cola_espera)
    pos_estudiante = buscar_x_legajo(lista_estudiantes, legajo_atender)
    estudiante = lista_estudiantes[pos_estudiante]
    print(f"Atendiendo consulta del estudiante: {estudiante['nombre']} (Legajo:
{estudiante['legajo']})")

""" Obtiene el mejor el promedio de una coleccion de notas"""
def mejor_promedio(lista_estudiantes):
    if len(lista_estudiantes) == 0:
        return None
    mejor_estudiante = lista_estudiantes[0]
    for estudiante in lista_estudiantes[1:]: #comparacion de hermanos
        if estudiante["promedio"] > mejor_estudiante["promedio"]:
            mejor_estudiante = estudiante
    return mejor_estudiante

""" Obtiene el peor el promedio de una coleccion de notas"""
def peor_promedio(lista_estudiantes):
```

```
if len(lista_estudiantes) == 0:
    return None
mejor_estudiante = lista_estudiantes[0]
for estudiante in lista_estudiantes[1:]: #comparacion de hermanos
    if estudiante["promedio"] > mejor_estudiante["promedio"]:
        mejor_estudiante = estudiante
return mejor_estudiante

""" Calcula el promedio de una coleccion de notas"""
def promedio_gral(lista_notas):
    return sum(lista_notas) / len(lista_notas) if len(lista_notas) > 0 else 0.0

""" Imprime estadisticas generales de la materia; informacion de sus alumnos"""
def estadisticas(lista_estudiantes, cola_espera):
    cantidad_estudiantes = len(lista_estudiantes)
    if cantidad_estudiantes == 0:
        print("No hay estudiantes para calcular estadísticas.")
        return

    print(f"Cantidad de estudiantes: {len(lista_estudiantes)}.")
    print(f"informaiocn de los estudiantes.")
    mostrar_estudiante(lista_estudiantes)

    lista_notas_generales = []
    for i, estudiante in enumerate(lista_estudiantes):
        lista_notas_generales.extend(estudiante["calificaciones"])

    print(f"Promedio general de la materia
{promedio_gral(lista_notas_generales)}")

    print(f"Mejor estudiante {mejor_promedio(lista_estudiantes)}")
    print(f"Peor estudiante {peor_promedio(lista_estudiantes)}")
    print(f"Cantidad de estudiantes en cola de consultas:
{tamano(cola_espera)}")

# BLOQUE PRINCIPAL, presenta el menu y ejecuta las opciones seleccionadas por el
usuario
lista_estudiantes = crear_lista_estudiante() # almacena la lista de estudiante .
cola_espera = crear_cola() #cola apara atender las consultas

while True:
    print("\n--- MENU ---")
    print("1. Agregar estudiante")
    print("2. Agregar calificación a estudiante")
    print("3. Agregar estudiante cola de consultas")
    print("4. Atender siguiente consulta")
    print("5. ver estudiantes y sus promedios")
    print("6. Ver estadisticas en general.")
    print("7. Salir")

    opcion = input("Seleccione una opción: ")

    if opcion == "1":
        ingresar_estudiante(lista_estudiantes)
    elif opcion=="2":
        registrar_nota(lista_estudiantes)
    elif opcion == "3":
```

```
        agregar_estudiante_cola(cola_espera, lista_estudiantes)
    elif opcion == "4":
        atender_consulta(cola_espera, lista_estudiantes)
    elif opcion == "5":
        mostrar_estudiante(lista_estudiantes)
    elif opcion == "6":
        print(f"Cantidad de estudiantes: {len(lista_estudiantes)}")
        if len(lista_estudiantes) > 0:
            promedio_general = sum(estudiante["promedio"] for estudiante in
lista_estudiantes) / len(lista_estudiantes)
            print(f"Promedio general de todos los estudiantes:
{promedio_general}")
        else:
            print("No hay estudiantes para calcular el promedio general.")

    elif opcion == "7":
        print("Saliendo del programa...")
        break
```